Anuprita Mhapankar **D20A** **34**

Blockchain Experiment 6

AIM: Creating Smart Contracts and performing transactions using Solidity and Remix IDE

THEORY:

Q1: What is a Smart Contract?

A smart contract is a self-executing digital agreement stored on a blockchain. It automatically runs when certain conditions written in the code are met. Smart contracts do not require any middleman like a bank or lawyer because the rules and execution are controlled by the program itself. Once the contract is deployed on the blockchain, it cannot be easily changed and all transactions are recorded permanently. This makes the process transparent, secure, and reliable.

Q2: Significance of smart Contracts in Ethereum Blockchain

Smart contracts play a very important role in the Ethereum blockchain.

- **Automation:** Transactions and agreements are executed automatically when conditions are met.
- **No Intermediaries:** Removes the need for third parties like banks or agents.
- **Transparency:** All contract details and transactions are recorded on the blockchain and can be verified.
- **Security:** Data stored on the blockchain is difficult to alter, making contracts more secure.
- **Cost Efficient:** Reduces transaction and processing costs by eliminating middlemen.
- **Used in Many Applications:** Examples include voting systems, financial services (DeFi), supply chain management, and digital agreements.

CODE:

Smart Contract 1: OrderRecord.sol

Records each order placed on the platform permanently on the blockchain. Stores the buyer's wallet address, a comma-separated list of product IDs, the total amount in paisa, and a timestamp.

```
//Smart Contract 1: OrderRecord.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract OrderRecord {
    struct Order {
        address buyer;
        string productIds;
```

```
        uint256 totalPaisa;
        uint256 timestamp;
    }

    Order[] public orders;

    event OrderPlaced(address buyer, string productIds, uint256
totalPaisa, uint256 timestamp);

    function placeOrder(string memory _productIds, uint256 _totalPaisa)
public {
        orders.push(Order(msg.sender, _productIds, _totalPaisa,
block.timestamp));
        emit OrderPlaced(msg.sender, _productIds, _totalPaisa,
block.timestamp);
    }

    function getOrderCount() public view returns (uint256) {
        return orders.length;
    }
}
```

Smart Contract 2: ReviewVerification.sol

Records product reviews on the blockchain to ensure authenticity and prevent tampering. Stores the product ID, reviewer's wallet address, star rating (1–5), and a timestamp. Also prevents the same wallet from reviewing the same product twice.

```
//Smart Contract 2: ReviewVerification.sol
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;

contract ReviewVerification {
    struct Review {
        string productId;
        address reviewer;
        uint8 rating;
        uint256 timestamp;
    }
```

```
    Review[] public reviews;
    mapping(string => mapping(address => bool)) public hasReviewed;

    event ReviewAdded(string productId, address reviewer, uint8 rating,
uint256 timestamp);

    function addReview(string memory _productId, uint8 _rating) public {
        require(_rating >= 1 && _rating <= 5, "Rating must be 1-5");
        require(!hasReviewed[_productId][msg.sender], "Already reviewed");

        reviews.push(Review(_productId, msg.sender, _rating,
block.timestamp));
        hasReviewed[_productId][msg.sender] = true;

        emit ReviewAdded(_productId, msg.sender, _rating,
block.timestamp);
    }

    function getReviewCount() public view returns (uint256) {
        return reviews.length;
    }
}
```

PROCEDURE

Step 1: Open Remix IDE

Open the Remix Integrated Development Environment in the browser. Remix IDE is used to write, compile, deploy, and test Solidity smart contracts.

https://remix.ethereum.org

Step 2: Create Smart contracts

a. **Order Record Contract**

Records every order placed through the application permanently on the blockchain.

Stores on blockchain: buyer wallet address, product IDs (comma-separated), total amount in paisa, and timestamp.

- buyer wallet address
- product IDs (comma-separated)
- total amount in paisa
- timestamp.

Benefit:

- Prevents tampering of order records
- Provides transparent and immutable proof of purchase

b. Review Verification Contract

Ensures that product reviews are genuine and cannot be altered after submission.

Stores on blockchain:

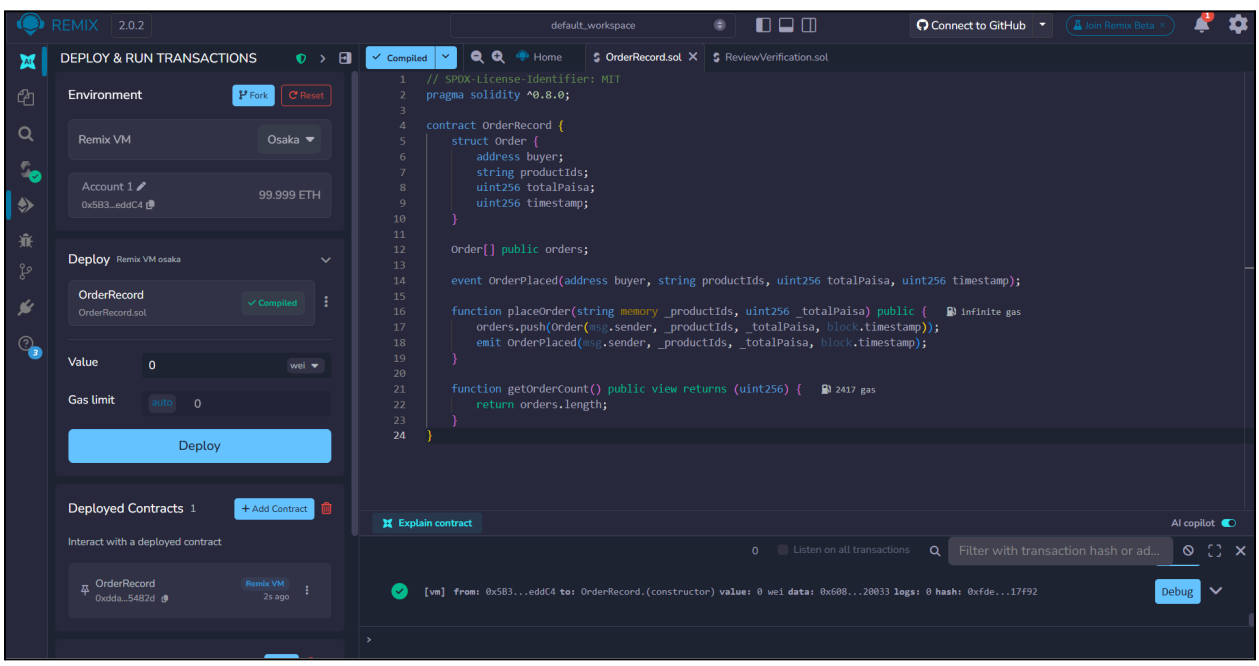
- product ID
- reviewer wallet address
- rating (1–5)
- timestamp.

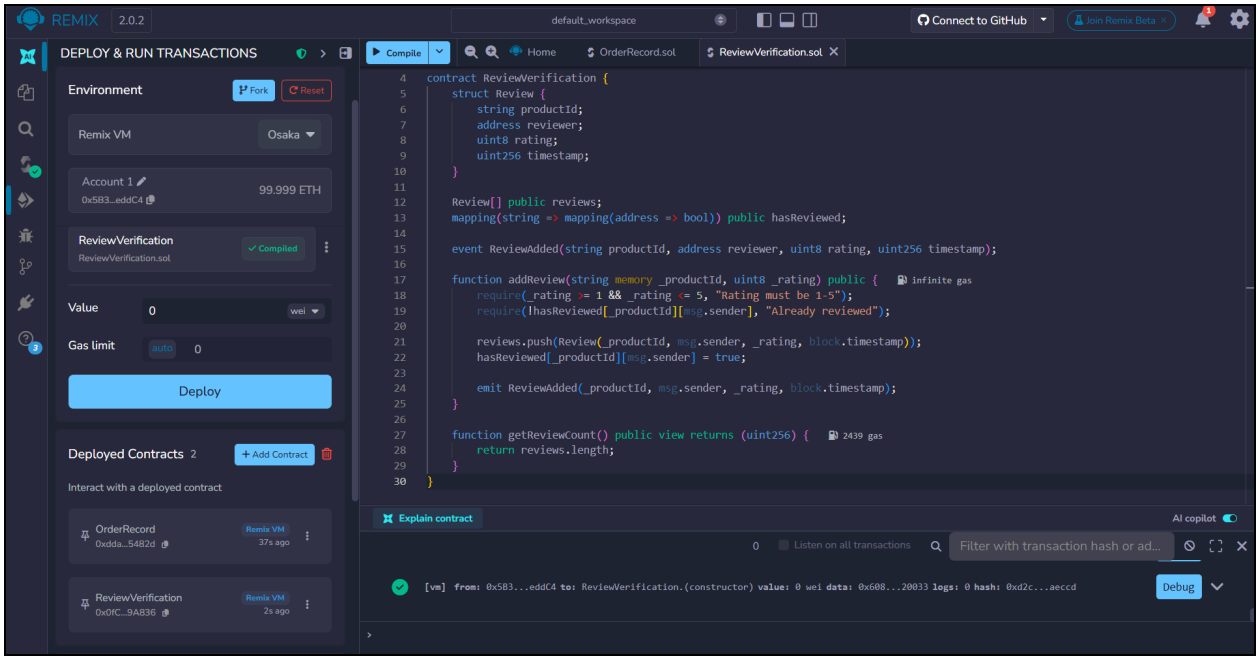
Benefit:

- Prevents fake or duplicate reviews from the same wallet
- Provides immutable, verifiable proof of each review

Step 3: Compile and deploy Smart contracts

Compile the smart contracts using the Solidity compiler in Remix IDE. After successful compilation, deploy the contracts on the Sepolia Test Network using the MetaMask wallet.

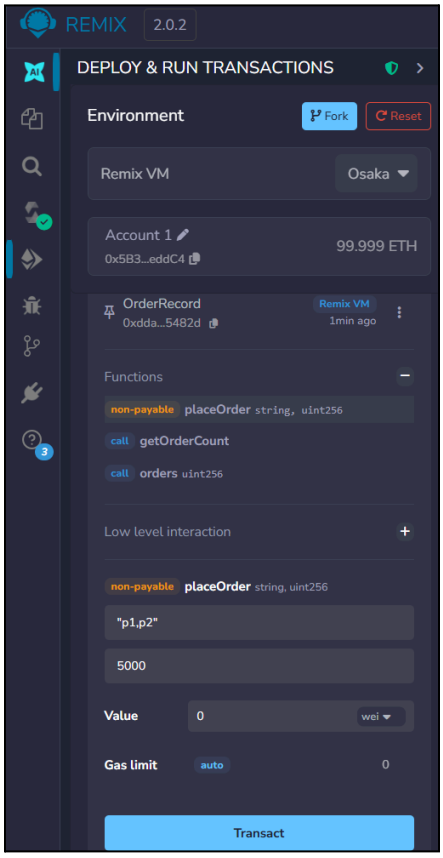


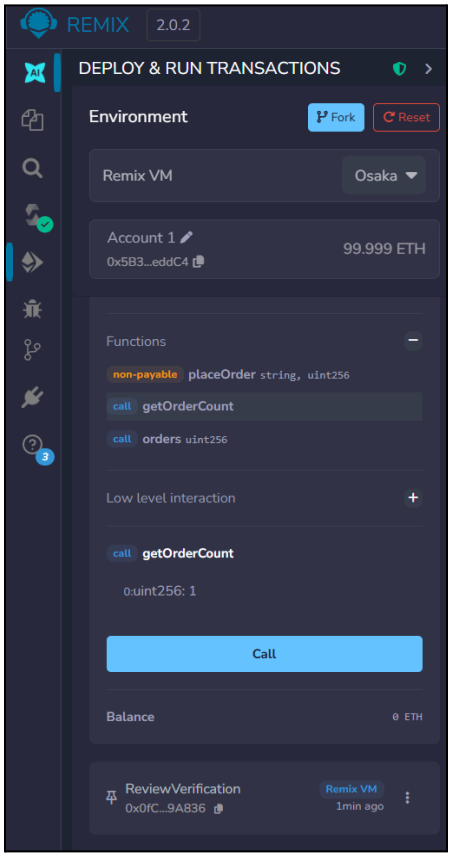


Step 4: Test the contracts

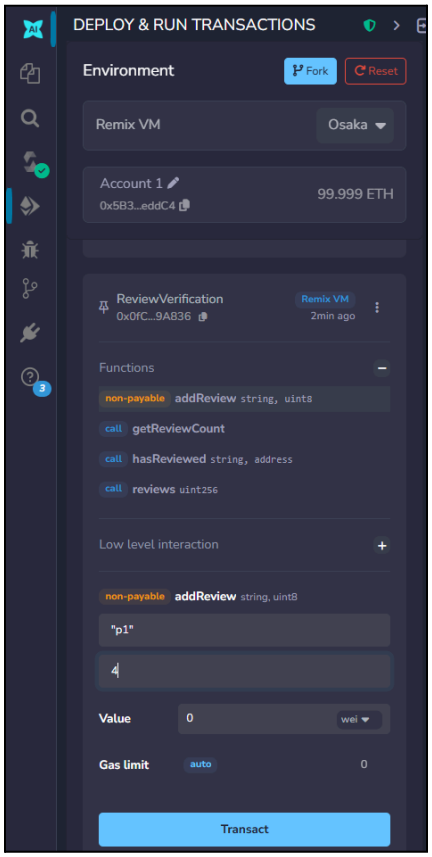
After deployment, test the contract functions such as addProduct, getProduct, and addReview. Verify that product details and reviews are stored correctly on the blockchain and can be retrieved using the contract functions.

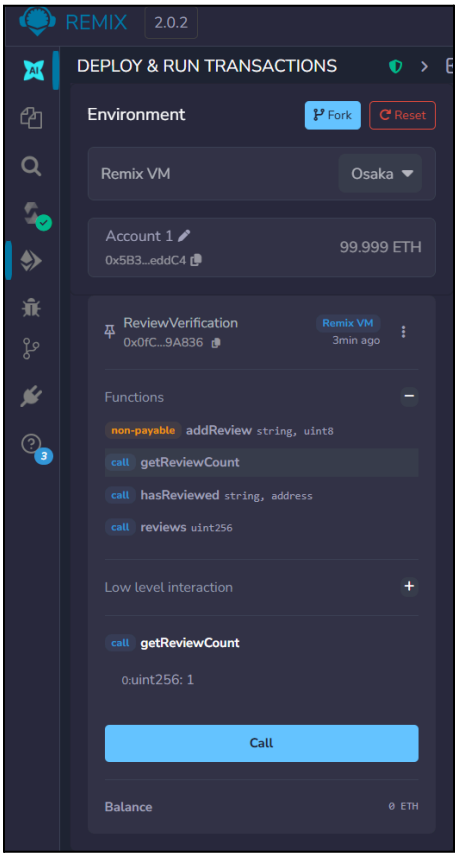
a. Order Record Contract





b. Review Verification Contract





CONCLUSION:

Through this experiment, two smart contracts - OrderRecord and ReviewVerification, were successfully created and deployed on the Ethereum Sepolia test network using Solidity and Remix IDE. The experiment demonstrated how blockchain technology can be integrated into a full-stack web application to permanently and transparently record order transactions and product reviews. The use of the Sepolia test network and MetaMask allowed safe testing without real funds. Overall, this experiment provided practical understanding of designing, compiling, deploying, and testing smart contracts on the Ethereum blockchain.